

Program-9: Organize Kubernetes pods using labels and namespaces

Step 1: Start Minikube

minikube start

```
harsha@HARSHA:~/program9$ minikube start
🐹 minikube v1.37.0 on Ubuntu 24.04
🌟 Using the docker driver based on existing profile
👍 Starting "minikube" primary control-plane node in "minikube" cluster
🚚 Pulling base image v0.0.48 ...
🔄 Restarting existing docker container for "minikube" ...
❗ Failing to connect to https://registry.k8s.io/ from inside the minikube container
💡 To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/proxy/
🐳 Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
🔍 Verifying Kubernetes components...
   ■ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌻 Enabled addons: storage-provisioner, default-storageclass
```

Step 2: Create Project Directory

mkdir Program-9

cd Program-9

Step 3: Create namespaces.yaml

nano namespaces.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
---
apiVersion: v1
kind: Namespace
metadata:
  name: prod
```

Step 4: Create pods.yaml

nano pods.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: frontend-pod
```

```
namespace: dev
labels:
  app: frontend
  env: dev
spec:
  containers:
  - name: frontend
    image: nginx
    ports:
    - containerPort: 80
```

```
apiVersion: v1
kind: Pod
metadata:
  name: backend-pod
  namespace: dev
  labels:
    app: backend
    env: dev
```

```
spec:
  containers:
  - name: backend
    image: nginx
    ports:
    - containerPort: 80
```

```
apiVersion: v1
kind: Pod
metadata:
  name: frontend-pod
  namespace: prod
  labels:
    app: frontend
    env: prod
```

```
spec:
  containers:
  - name: frontend
    image: nginx
    ports:
    - containerPort: 80
```

```
apiVersion: v1
kind: Pod
metadata:
  name: backend-pod
  namespace: prod
  labels:
    app: backend
```

```
  env: prod
spec:
  containers:
  - name: backend
    image: nginx
    ports:
    - containerPort: 80
```

Step 5: Apply Namespace YAML

kubectl apply -f namespaces.yaml

```
harsha@HARSHA:~/program9$ kubectl apply -f namespaces.yaml
namespace/dev unchanged
namespace/prod unchanged
```

Step 6: Apply Pod YAML

kubectl apply -f pods.yaml

```
harsha@HARSHA:~/program9$ kubectl apply -f pods.yaml
pod/frontend-pod created
pod/backend-pod created
pod/frontend-pod created
pod/backend-pod created
```

step 7: Verify Pods by Namespace

kubectl get pods -n dev

kubectl get pods -n prod

```
harsha@HARSHA:~/program9$ kubectl get pods -n dev
kubectl get pods -n prod
NAME                READY   STATUS    RESTARTS   AGE
backend-pod         1/1     Running   0           21s
frontend-pod        1/1     Running   0           21s
NAME                READY   STATUS    RESTARTS   AGE
backend-pod         1/1     Running   0           21s
frontend-pod        0/1     ContainerCreating   0           21s
```

Step 8: Check Pod Status

kubectl get pods --all-namespaces

```
harsha@HARSHA:~/program9$ kubectl get pods --all-namespaces
```

NAMESPACE	NAME	READY	STATUS	RESTARTS
default	nodejs-pod	1/1	Running	3 (2m44s ago)
dev	backend-pod	1/1	Running	0
dev	frontend-pod	1/1	Running	0
kube-system	coredns-66bc5c9577-gsbpz	1/1	Running	3 (10h ago)
kube-system	etcd-minikube	1/1	Running	3 (10h ago)
kube-system	kube-apiserver-minikube	1/1	Running	3 (2m44s ago)
kube-system	kube-controller-manager-minikube	1/1	Running	4 (10h ago)
kube-system	kube-proxy-hfxqx	1/1	Running	3 (10h ago)
kube-system	kube-scheduler-minikube	1/1	Running	3 (10h ago)
kube-system	storage-provisioner	1/1	Running	6 (2m1s ago)

Step 9: Filter Pods Using Labels

List frontend pods in dev namespace

List backend pods in prod namespace

```
harsha@HARSHA:~/program9$ kubectl get pods -n dev -l app=frontend
```

NAME	READY	STATUS	RESTARTS	AGE
frontend-pod	1/1	Running	0	54s

```
harsha@HARSHA:~/program9$ kubectl get pods -n prod -l app=backend
```

NAME	READY	STATUS	RESTARTS	AGE
backend-pod	1/1	Running	0	61s

```
harsha@HARSHA:~/program9$
```

Step 10: Clean Up Pods

kubectl delete pods --all -n dev

kubectl delete pods --all -n prod